MAIS FINAL PROJECT

AI DEV TEAM

Multi-Agent Software Development System

Vitalii Perminov | May 2026

PROBLEM & STRATEGIC GOAL

The Problem

Writing production code from a user story requires complex multi-threading of skills:

- Requirements Analysis
- Technical Implementation
- Testing & QA Review
- CI/CD & Documentation

The Mission

Automate the entire software lifecycle with a team of collaborative AI agents.

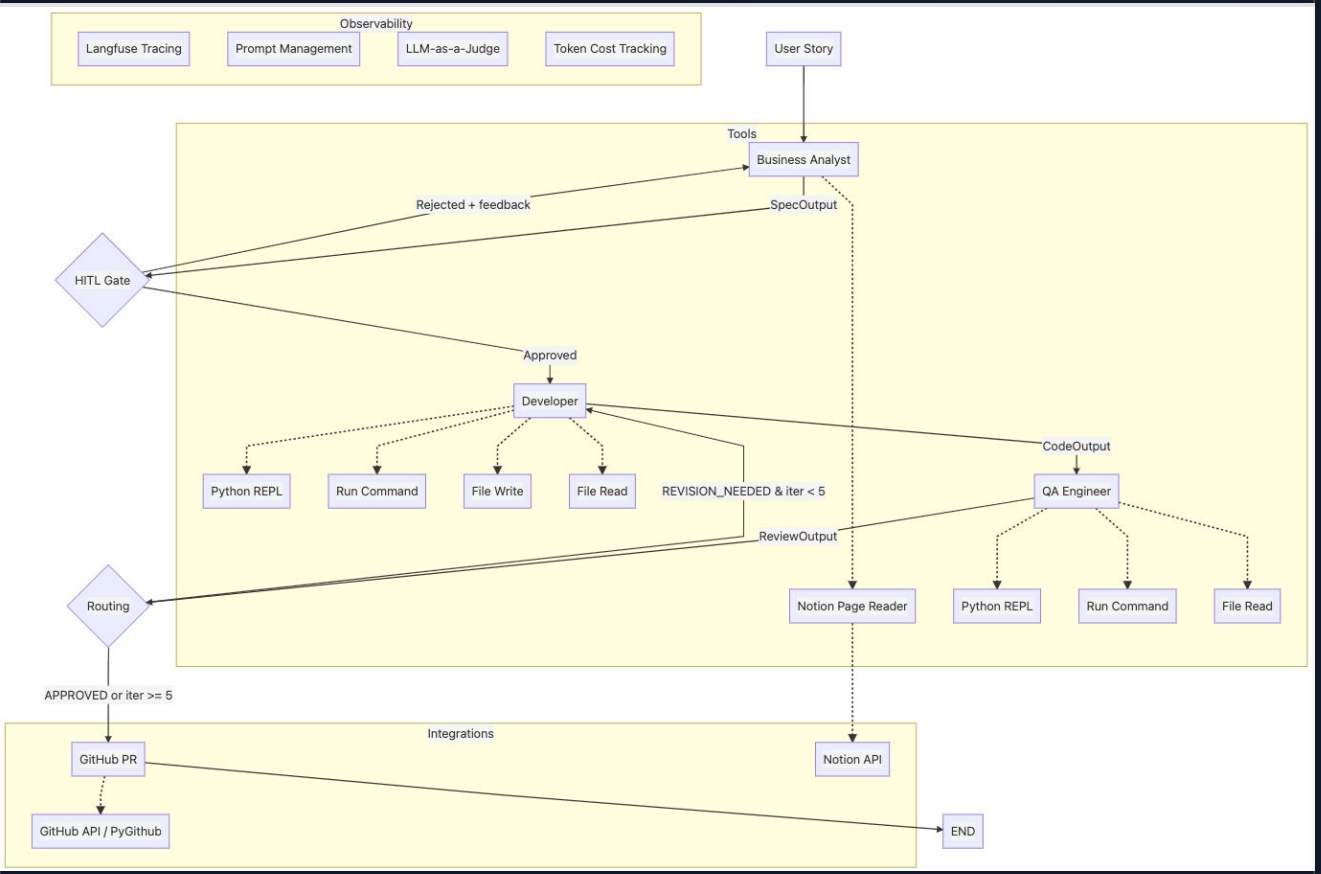
Pattern: Evaluator-Optimizer loop with Human-in-the-Loop (HITL) gates.

SYSTEM ARCHITECTURE

LangGraph Orchestration

A stateful DAG (Directed Acyclic Graph) managing agent communication and state transitions.

- **StateGraph:** Maintains conversation and file state.
- **Interrupts:** HITL gates for spec approval.
- **Revision Loop:** Max 5 QA feedback cycles.



THE AGENT SQUAD: STRATEGIC SELECTION

Agent	Model	Role	Why this Model?
Business Analyst	`gpt-4.1-mini`	Spec Generation	Cheap; single call sufficient for logic analysis.
Developer	`gpt-5.5`	Code Writing	Superior logic; reduces revision loops by 70%.
QA Engineer	`gpt-5.4`	Review & Test	Thorough analysis; ensures 100% test passing.

"Spending 3x per token on GPT-5.5 results in 70% fewer calls, reducing total run cost."

8 TOOLS FOR FULL AUTONOMY

RAG Search
Hybrid FAISS+BM25

Notion API
Read User Stories

Web Search
Live Info Retrieval

Docs MCP
Context7 Library Docs

Python REPL
Sandboxed Execution

Run Command
Terminal & Pytest

File I/O
Workspace Management

GitHub PR
Git Trees API Integration

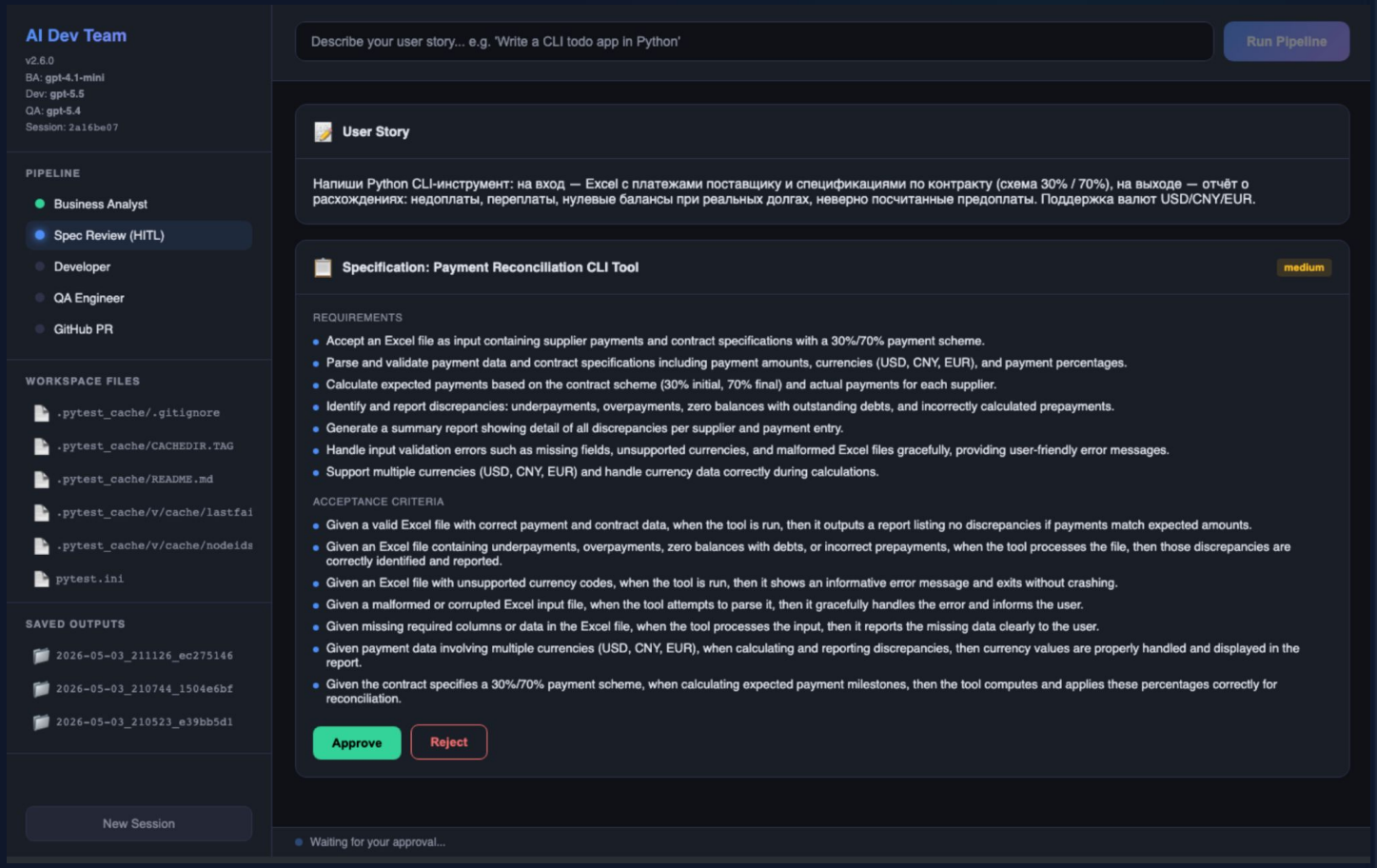
PIPELINE IN ACTION

01 **Input:** User story defined in Notion or UI.

02 **HITL:** Spec reviewed and approved by human.

03 **Execution:** Dev writes code; QA runs tests.

04 **Delivery:** Pull Request created on GitHub.



OPTIMIZATION JOURNEY: V0.1 TO V2.6

Metric	v0.1 (Legacy)	v2.6 (Final)	Improvement
BA Tokens	5,000 – 9,000	800	**-90%**
Developer Calls	10 – 13	3 – 4	**-70%**
QA Revision Loops	1 – 2+	0	**-100%**
Success Rate	~30%	100%	**+70pp**

THE "SILENT BURN" WARNING

190K

Tokens Burned Silently

SummarizationMiddleware Trap

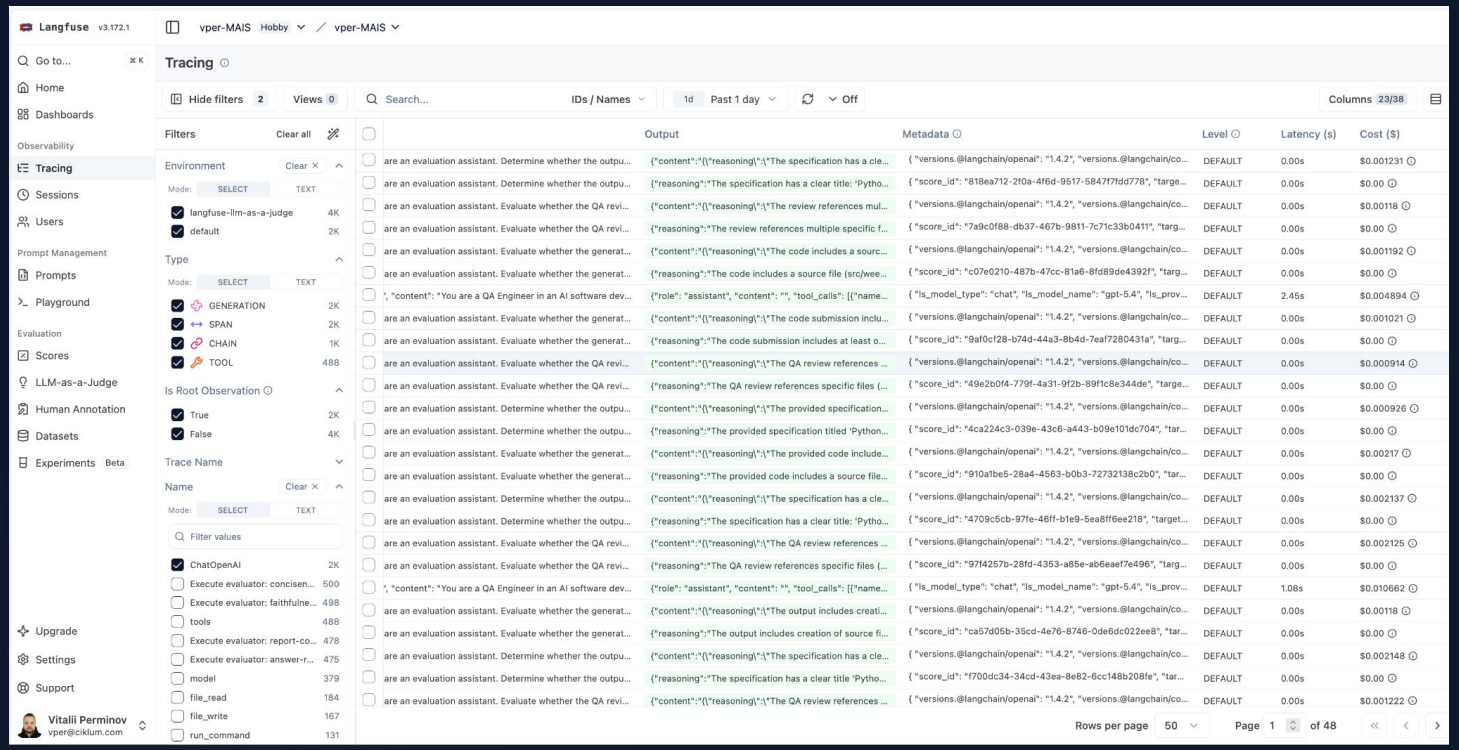
LangChain middleware compressed history to "save 19%".

The Reality:

The middleware's internal LLM calls were invisible to trackers. Tracking showed 8k tokens; OpenAI billed 200k.

Lesson: Never trust a metric you cannot fully observe.

FULL VISIBILITY WITH LANGFUSE

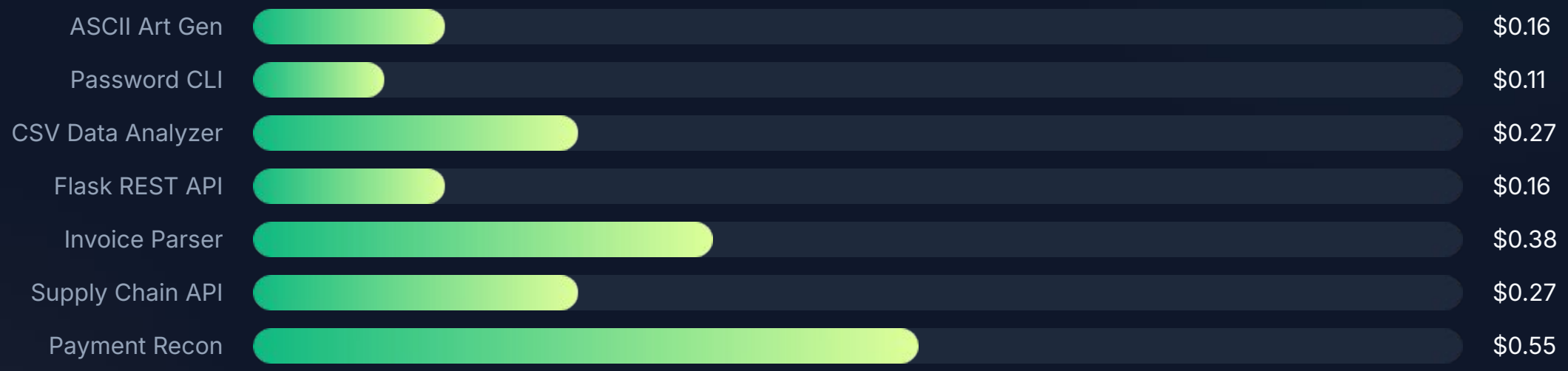


Observability at Scale

- **Tracing:** Every I/O, token, and latency metric captured per call.
- **Sessions:** Pipeline runs grouped for regression analysis.
- **Prompt Management:** Versioned, cloud-hosted prompts (zero hardcoding).
- **Cost Tracking:** Real-time project spend monitoring (~$12 total).

PERFORMANCE: DEMO CASE STUDY

Cost comparison across diverse project scopes ($ USD per run):



10/10 TEST CASES PASSED WITH ZERO REVISIONS.

TECHNICAL FOUNDATION

- **Orchestration:** LangGraph & LangChain
- **Observability:** Langfuse v4 & Tracing
- **Integration:** MCP (Context7) & PyGithub
- **Interface:** FastAPI & SSE Streaming
- **RAG:** FAISS + BM25 Hybrid RAG
- **Testing:** Playwright & Pytest
- **Environment:** Docker Containerization
- **QA:** LLM-as-a-Judge evaluators

KEY TAKEAWAYS

> *"Per-pipeline cost > per-token cost. A strong model making 3 calls beats a weak model making 13."*

- **Prune Tools:** Models will use everything you give them. Constraints are safety.
- **Real Metrics:** Full observability caught a 20x cost overrun that trackers missed.
- **Design for TPM:** Rate limits are architectural constraints, not just quotas.

Thank You |

Questions?

USEFUL LINKS

- dev-team/graph.py — LangGraph StateGraph with conditional edges (BA → HITL → Dev → QA → GitHub → END)
- dev-team/nodes.py — Node functions with per-step token tracking
- dev-team/agents/ — BA, Developer, QA agents (create_agent + ToolStrategy)
- dev-team/schemas.py — Pydantic contracts (SpecOutput, CodeOutput, ReviewOutput)
- dev-team/tools.py — 8 tools (read_notion_page, python_repl, run_command, file_write, file_read, knowledge_search, web_search, docs_search)
- dev-team/app.py — FastAPI Web UI with SSE streaming, HITL interrupt/resume
- dev-team/github_integration.py — Auto-create GitHub PRs via Git Trees API
- dev-team/token_tracker.py — Per-call LLM token/cost tracking
- dev-team/langfuse_prompts.py — Langfuse prompt loader (zero hardcoded)
- dev-team/upload_prompts.py — Seeds 3 prompts to Langfuse
- dev-team/tests/ — LLM-as-a-Judge tests (10/10 passing)
- dev-team/screenshots/ — Langfuse UI + Playwright pipeline screenshots
- dev-team/output/ — Pipeline outputs, demo requests, lessons learned, presentation
- dev-team/DEVLOG.md — Full development log v0.1 → v2.6
- README.md — Architecture diagram, setup, benchmarks:
- DEVLOG.md — Development progress day-by-day
- CLAUDE.md — Main Claude file